A Trimester – 1 Class Project Report on

FILE ORGANISER AND DUPLICATION DETECTION TOOL - FilePilot

Submitted in the partial fulfillment of the requirements for

26CS5101E – Computational Thinking and Programming

Master of Technology In

Artificial Intelligence & Data Science

By

Student ID	Student Name
2601150019	T. NAGA PRANAV

under the supervision of

Dr. G. Syam Prasad

Professor

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd., u/s 3 of UGC Act 1956)

Greenfields, Vaddeswaram, Guntur (Dist.), Andhra Pradesh - 522502

2026 – 27

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



CERTIFICATE

This is to certify that the Trimester – 1 Class Project Report entitled "**File Organizer and Duplication Detection Tool - FilePilot**" is being submitted by T. Naga Pranav - 2601150019 submitted in partial fulfillment the requirements for the course 26CS5101E – Computational Thinking and Programming, Master of Technology in Artificial Intelligence and Data Science to the K L University is a record of Bonafide work carried out under our guidance and supervision during the academic year 2026 – 2027.

Dr. G. Syam Prasad

Project Supervisor

Signature

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



DECLARATION

The Trimester -1 Class Project Report entitled "**File Organizer and Duplication Detection Tool - FilePilot**" is a record of Bonafide work of T. Naga Pranav - 2601150019 submitted in partial fulfillment the requirements for the course 26CS5101E – Computational Thinking and Programming, Master of Technology in Artificial Intelligence and Data Science to the K L University.

TUMMALAPALLI NAGA PRANAV - 2601150019

ACKNOWLEDGEMENT

It is great pleasure for me to express my gratitude to our honorable President **Sri. Koneru Satyanarayana**, for giving the opportunity and platform with facilities in accomplishing the trimester - 1 class project report.

I express sincere gratitude to HOD of AI&DS Dept. **Dr. M. Venkata Naresh** for his leadership and constant motivation provided in successful completion of our academic semester. I record it as my privilege to deeply thank for providing us the efficient faculty and facilities to make our ideas into reality.

I express my sincere thanks to our project supervisor **Dr. G. Syam Prasad** for his novel association of ideas, encouragement, appreciation, and intellectual zeal which motivated us to venture this project successfully.

Finally, it is pleased to acknowledge the indebtedness to all those who devoted themselves directly or indirectly to make this project report success.

ABSTRACT

FilePilot is a local-first file organization and duplicate detection application developed using Python, Flask, SQLite, HTML, CSS, JavaScript, and SHA-256 hashing. It helps users inspect a selected directory, view file metadata, identify exact duplicate files, understand file-type distribution, and organize top-level files into category folders. The application provides a preview-before-action workflow so users can review proposed moves before executing them. Duplicate files are reported rather than automatically deleted. The dashboard presents summary cards, charts, a searchable inventory, and scan history. The project applies computational thinking through problem decomposition, abstraction, pattern recognition, algorithm design, data structures, and complexity analysis. Safety measures include skipping symbolic links, avoiding destination-name overwrites, and keeping the web server bound to localhost. The implementation also includes unit tests for duplicate grouping, preview safety, organization behavior, and unknown file extensions.

INDEX

S. No	TITLE	PAGE NO
1	Introduction	1
2	Problem Statement	2
3	System Architecture and Technologies	3
4	Problem Solving & Computational Thinking	5
5	Programming & Data Structures	7
6	Software Design & Engineering	8
7	Functional Programming & Concurrency	9
8	Software Quality & DevOps Practices	10
9	AI-Assisted Software Engineering	12
10	Algorithms, Complexity and Performance	13
11	Implementation and User Workflow	15
12	Testing, Results and Limitations	16
13	Conclusion and Future Enhancements	19
14	References	20

1. INTRODUCTION

FilePilot is a browser-based tool designed to make local file management more systematic and safer. It scans a user-selected directory, creates an inventory of accessible files, detects exact duplicates, summarizes file categories, and offers a controlled workflow for organizing files. The interface is served by Flask and is intended for local use rather than public deployment.

The project is relevant to Computational Thinking and Programming because it converts a practical problem into a sequence of smaller tasks: directory traversal, metadata extraction, hashing, classification, aggregation, visualization, organization, and history storage. Each task has a defined responsibility and can be tested separately.

1.1 Objectives:

- Scan a selected directory and report accessible file metadata.
- Classify files into Images, Documents, Audio, Video, Archives, Code, and Other.
- Detect exact duplicate content using file size and SHA-256 hashes.
- Present file counts, storage totals, duplicate groups, and charts in a dashboard.
- Preview proposed file moves and avoid overwriting existing destination names.
- Store recent scan summaries in a local SQLite database.

2. PROBLEM STATEMENT

Personal workspaces and download folders often contain mixed file types, unclear names, and repeated copies. Manually sorting files takes time. Deleting duplicates without checking their contents can also cause data loss. A useful solution should first provide an accurate inventory, distinguish exact duplicates from files that merely have similar names, and let the user review changes before any move is performed.

2.1 Proposed Solution:

FilePilot provides a local web dashboard for scanning a chosen directory and displaying file metadata, duplicate groups, category summaries, extension statistics, and recent scan history. It also provides an organization preview that maps top-level files to category folders. The user can inspect the planned source and destination paths before executing the move.

2.2 Scope:

Inventory and duplicate analysis traverse directories recursively, subject to filesystem access permissions. The organization operation moves only files directly inside the selected root directory; files inside nested folders are not rearranged. The application identifies exact byte-for-byte content matches and does not automatically delete duplicate files.

2.3 Expected Outcomes:

The expected outcome is a working local application that improves visibility into a folder's contents, identifies exact duplicates, and reduces manual file sorting. The supplied project includes unit tests for core file-processing behavior. Actual performance depends on file count, total bytes read, storage speed, and filesystem permissions.

3. SYSTEM ARCHITECTURE AND TECHNOLOGIES

The application uses a browser-based frontend, a Flask backend, a file-processing module, the operating-system filesystem, and a local SQLite database. The browser sends requests to Flask routes. The backend invokes file-processing functions, prepares summary data, and renders templates or returns JSON.

3.1 Architecture Flow:

- Browser dashboard: HTML templates, CSS styling, JavaScript interactions, and Chart.js visualizations.
- Flask application (app.py): dashboard route, organization and history routes, summary calculations, persistence, and JSON API.
- File-processing module (file_tools.py): directory traversal, metadata extraction, extension-based classification, chunked hashing, duplicate grouping, and safe file moves.
- Filesystem: the selected directory and the category folders created by the organization workflow.
- SQLite database (organizer.db): recent scan summaries used by the history page.

3.2 Technology Stack:

- Python: implements the core processing logic and application backend.
- Flask: provides local web routes and connects the processing layer to the interface.
- SQLite: stores scan history without requiring a separate database server.
- HTML and CSS: structure and style the dashboard, preview page, and history page.
- JavaScript and Chart.js: render charts and support client-side dashboard interactions.
- hashlib / SHA-256: produces content digests used for exact duplicate grouping.
- unittest: runs automated checks for core file-processing behavior.

3.3 Main Modules:

app.py contains Flask routes, dashboard summary calculations, history persistence, and the scan API. file_tools.py contains reusable functions for category mapping, readable size formatting, chunked SHA-256 hashing, scanning, duplicate grouping, and organization. The templates directory contains the dashboard, organization preview, and history pages. The static directory contains styling and dashboard JavaScript. The tests directory contains test_file_tools.py.

4. PROBLEM SOLVING & COMPUTATIONAL THINKING

The project applies the computational thinking process by converting a broad file-management problem into smaller, testable tasks and selecting algorithms that suit each task.

4.1 Problem Decomposition:

The system separates scanning, hashing, categorization, duplicate grouping, visualization, file organization, and scan-history persistence. This separation makes the code easier to understand and allows the core file-processing functions to be tested independently.

4.2 Abstraction:

Instead of repeatedly processing every filesystem detail in the dashboard, the application represents each file as a compact record containing fields such as name, path, extension, category, size, modification time, and hash. The dashboard uses these records to calculate summaries and populate the inventory.

4.3 Pattern Recognition:

The duplicate detector groups records by file size and SHA-256 digest. Matching file names are not required. For example, two files named report.pdf and report_copy.pdf can be recognized as duplicates when their content bytes are identical. Files with different content are not grouped merely because their names are similar.

4.4 Algorithm Selection:

Directory traversal is used to discover files; chunked hashing limits the amount of file content held in memory at one time; dictionaries group matching size-and-hash pairs; sorting helps display larger duplicate groups first; and extension mapping assigns categories. The project does not require dynamic programming, backtracking, or branch-and-bound because its core tasks do not present the optimization problems those methods are designed to solve.

4.5 Complexity Analysis:

Let F be the number of files examined and B the total number of bytes read for hashing. Directory traversal and per-file metadata processing are approximately O(F), while hashing is approximately O(B). Grouping records by a dictionary key has expected O(F) time after hashes are computed. Sorting records or duplicate groups can add O(F log F) time. Organization planning examines N top-level entries and is approximately O(N), excluding filesystem move costs.

4.6 Data Safety as a Design Constraint:

The application prioritizes preview and confirmation over destructive automation. It skips symbolic links, avoids overwriting existing destination names by adding numbered suffixes, and does not automatically delete duplicate files. Users should still back up important data and test organization on disposable sample folders.

5. PROGRAMMING & DATA STRUCTURES

Python is used for filesystem access, file metadata handling, content hashing, categorization, grouping, database interaction, and the web backend. The implementation uses built-in data structures and standard library functionality to keep the solution straightforward and maintainable.

5.1 Data Structures:

- Lists hold collections of file records and duplicate groups for display and sorting.

- Dictionaries map file extensions to categories and group records using keys such as (file size, SHA-256 digest).

- File records organize metadata into a consistent structure that can be used by the dashboard and API.

- SQLite tables persist scan summaries across application visits.

5.2 Chunked File Reading:

The hashing routine reads files in chunks instead of loading an entire file into memory. This approach keeps memory use more stable for large files. Hashing still requires reading the accessible bytes, so the total processing time grows with the amount of data scanned.

5.3 Category Mapping:

Common image extensions such as .png, .jpg, .jpeg, .gif, and .webp map to Images. Extensions such as .pdf, .docx, .txt, .xlsx, and .csv map to Documents. Audio, video, archive, and code extensions map to their respective categories. Unrecognized extensions map to Other.

5.4 Performance-Aware Programming:

The project avoids reading an entire file into memory for hashing and uses dictionary-based grouping rather than comparing every file against every other file. This is more suitable for larger collections than a naive pairwise comparison, although actual performance should be measured on representative folders.

6. SOFTWARE DESIGN & ENGINEERING

The application is organized into separate modules for web routing, file processing, templates, static assets, and tests. This modular structure reduces coupling between the interface and filesystem algorithms.

6.1 Separation of Responsibilities:

The Flask layer handles requests and prepares responses, while file_tools.py performs the core filesystem work. HTML templates display the resulting data, and JavaScript handles chart rendering and client-side table search. SQLite is used for scan history rather than as a store for file contents.

6.2 Reusability and Maintainability:

Reusable functions for classification, size formatting, hashing, scanning, and organization allow common behavior to be invoked from different routes. The project uses a clear module boundary between application routing and file operations. The available implementation does not establish that every function uses type hints, inheritance, or a formal design pattern; these should not be claimed unless they are verified in the source.

6.3 Error Handling and File-System Constraints:

The scanner is designed to report access errors and continue scanning other files where possible. Destination name collisions are handled by generating a numbered alternative name rather than overwriting the existing file. Symbolic links are skipped to reduce the risk of following unexpected paths.

7. FUNCTIONAL PROGRAMMING & CONCURRENCY

The project primarily uses sequential filesystem processing. Its workflow can be understood as a sequence of transformations: discover files, convert them into metadata records, classify records, compute hashes, group duplicate candidates, and aggregate statistics for display.

7.1 Functional Decomposition:

Keeping operations such as extension classification, size formatting, and duplicate grouping in reusable functions makes the processing pipeline easier to reason about and test. These functions should only be described as pure functions when they do not read or modify external state; filesystem scanning and moving files are inherently side-effecting operations.

7.2 Concurrency Assessment:

The supplied project does not establish a threaded, multiprocessing, or asynchronous scanning implementation. Therefore, concurrency is not claimed as an implemented feature. File scanning and hashing are I/O-heavy, and a future version could evaluate background tasks or controlled concurrency to keep the dashboard responsive. Any such change would require safeguards against simultaneous moves, database conflicts, and duplicate scans.

7.3 GIL and Workload Considerations:

File reading is I/O-bound, while SHA-256 computation consumes CPU time as bytes are processed. Python's Global Interpreter Lock affects CPU-bound Python bytecode, but it does not mean all I/O waits block all threads. A performance improvement should be selected based on profiling rather than adding concurrency without evidence.

8. SOFTWARE QUALITY & DEVOPS PRACTICES

Software quality is addressed through modular implementation and unit tests for the core file operations. The project includes a tests directory and uses Python's unittest framework.

8.1 Test Coverage in the Supplied Project:

- Duplicate detection: identical file contents are grouped as duplicates.
- Preview safety: preview mode does not move files.
- Organization execution: a test file is moved into its expected category.
- Unknown extension handling: unsupported extensions are assigned to Other.

8.2 Additional Recommended Tests:

- Empty directories and directories containing nested folders.
- Unreadable files and permission errors.
- Large files and paths containing spaces.
- Destination name collisions and pre-existing category folders.
- Files with matching names but different content.
- Files with identical content but different names or extensions.

8.3 Execution and Dependencies:

The project requires Python 3.10 or newer. From the extracted project directory, install dependencies with pip install -r requirements.txt and launch the application with python app.py. The actual app.py configuration in the supplied project uses port 6969, so the browser address is http://127.0.0.1:6969. This differs from the port 5000 stated in the presentation/reference report; the source code should be treated as authoritative unless the port is changed.

Run the tests with python -m unittest discover -s tests -v. The application creates or uses organizer.db locally for scan history. The supplied project does not establish a configured CI/CD pipeline, containerization workflow, code-coverage report, or static type-checking pipeline, so these are not reported as completed DevOps features.

8.4 Local Security:

The application is configured for localhost use. It should not be exposed directly to a public network because it accepts filesystem paths and performs file operations. Users should use sample data first and retain backups before executing moves.

9. AI-ASSISTED SOFTWARE ENGINEERING

AI-assisted software engineering is one of the areas listed in the Project Defence requirements. For this project, AI tools may support requirement clarification, code explanation, test-case design, debugging, and documentation. However, use of a particular AI tool or generated code must be described truthfully based on the development process actually followed.

9.1 Specification-First Development:

A specification-first workflow begins by defining the required behavior: scan a chosen directory, capture file metadata, detect exact duplicates, preview organization, move files safely after confirmation, and retain scan summaries. These requirements provide criteria against which implementation and test results can be checked.

9.2 Review and Validation:

AI-generated suggestions should not be accepted without review. File movement and path-handling code can damage a user's folder structure if assumptions are wrong. Suggested changes should be checked against the code, run on disposable sample folders, and covered by tests. The supplied project contains unit tests for key file-processing behaviors.

9.3 Responsible Use:

AI assistance should be documented according to the work actually performed. This report does not claim specific prompts, AI-generated modules, or AI-tool validation sessions as completed activities because those details are not evidenced by the project files.

10. ALGORITHMS, COMPLEXITY AND PERFORMANCE

10.1 Duplicate Detection Algorithm:

- Traverse the selected directory and skip symbolic links.
- Collect metadata for each accessible file.
- Read file contents in chunks and calculate the SHA-256 digest.
- Use file size and digest as the grouping key.
- Keep groups containing more than one file as duplicate groups.
- Sort duplicate groups so larger groups can be reviewed first.

10.2 File Organization Algorithm:

- Inspect only direct files inside the selected root for the organization operation.
- Map each file extension to a destination category.
- Construct a proposed destination path and check for name collisions.
- Display the source and destination in preview mode without moving files.
- On execution, move the file and use a numbered suffix if the destination name already exists.

10.3 Performance Considerations:

Hashing requires reading all accessible file bytes, so very large files and large directories can increase scan time. Chunked reads reduce peak memory requirements. Dictionary-based grouping avoids repeated all-pairs content comparisons. No numeric benchmark is claimed in this report because the supplied materials do not provide a controlled timing experiment.

10.4 Storage Summary:

The dashboard calculates total scanned size and estimates the additional bytes represented by duplicate copies. This value is an estimate of redundant content in duplicate groups; it should not be interpreted as storage already reclaimed because the application does not delete duplicates.

11. IMPLEMENTATION AND USER WORKFLOW

11.1 Installation:

- Install Python 3.10 or newer.
- Extract the project ZIP and open the project folder in a terminal or VS Code.
- Optionally create and activate a virtual environment.
- Install packages using pip install -r requirements.txt.
- Start the application using python app.py.
- Open http://127.0.0.1:6969 in a browser.

11.2 Scanning a Folder:

The user provides a folder path through the dashboard. The application scans accessible files and builds an inventory containing file metadata. It then calculates summary information, category distribution, extension counts, and duplicate groups. Recent scan summaries are saved in SQLite and can be viewed through the history page.

11.3 Organizing Files Safely:

The user reviews the organization preview to see proposed source and destination paths. Preview mode is read-only with respect to file movement. If the plan is correct, the user can execute the operation on top-level files. Files in nested directories remain untouched, and destination collisions receive numbered suffixes. The application does not delete duplicate files.

11.4 Dashboard and API:

The dashboard combines summary cards, a category doughnut chart, an extension bar chart, and a searchable file inventory. The /api/scan endpoint provides scan information in JSON format for integration or inspection. The /history page displays recent saved scan summaries.

12. TESTING, RESULTS AND LIMITATIONS

12.1 Recorded Unit-Test Results:

The supplied test-results file records four successful tests: duplicate grouping, preview safety, organization execution, and unknown-extension categorization. The output also contains an environment startup traceback before the successful test summary. These results are reported as recorded in the supplied artifact; they have not been independently rerun as part of preparing this report.

12.2 Validation Table:

Test case	Expected behavior	Recorded status
Identical content files	One duplicate group is reported	Passed in supplied test output
Preview mode	Source files remain unmoved	Passed in supplied test output
Execute organization	File moves to expected category	Passed in supplied test output
Unknown extension	File is classified as Other	Passed in supplied test output

12.3 Limitations:

• Only exact byte-for-byte duplicates are detected; near-duplicate images and documents are outside scope.

• Hashing reads accessible file contents and may take time for large datasets.

• Organization moves only top-level files and does not rearrange nested folder contents.

• The application is intended for localhost use and should not be exposed publicly.

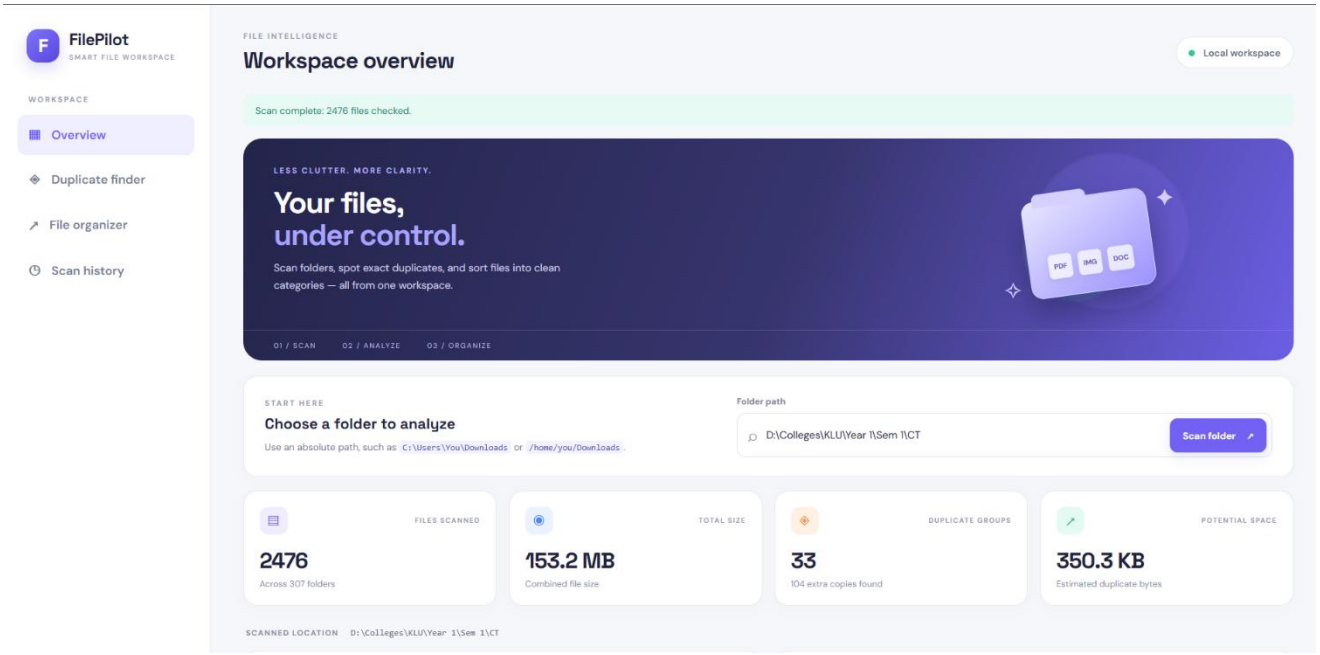
• The supplied evidence does not include numeric performance benchmarks, code-coverage results, or CI/CD configuration.

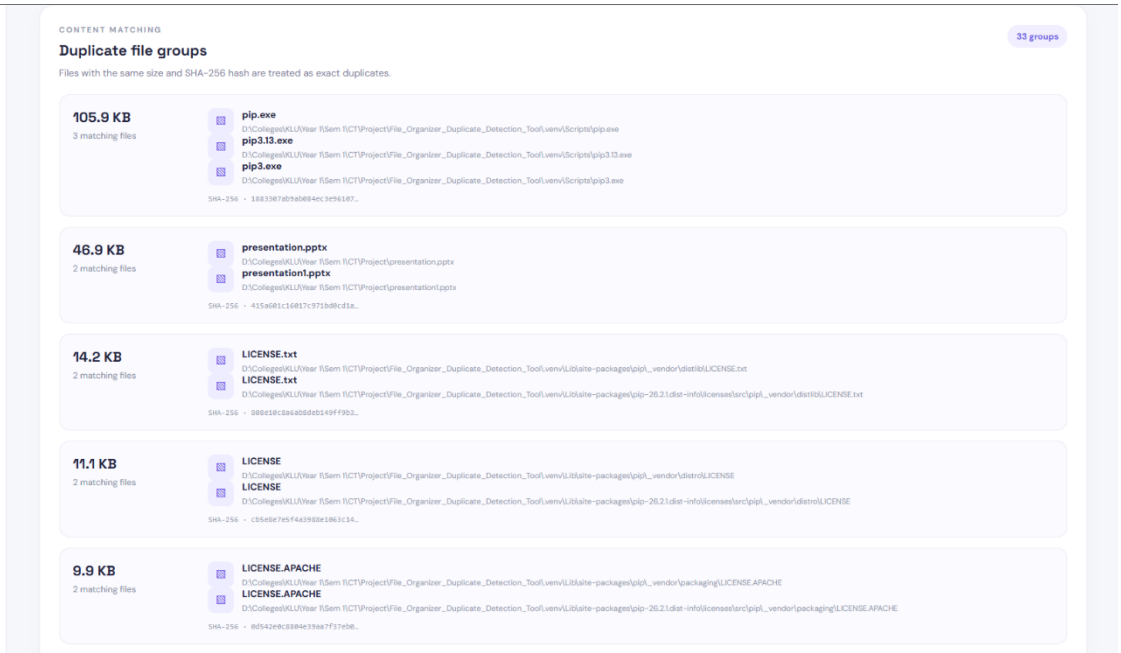
• External frontend resources may require an internet connection unless bundled locally.

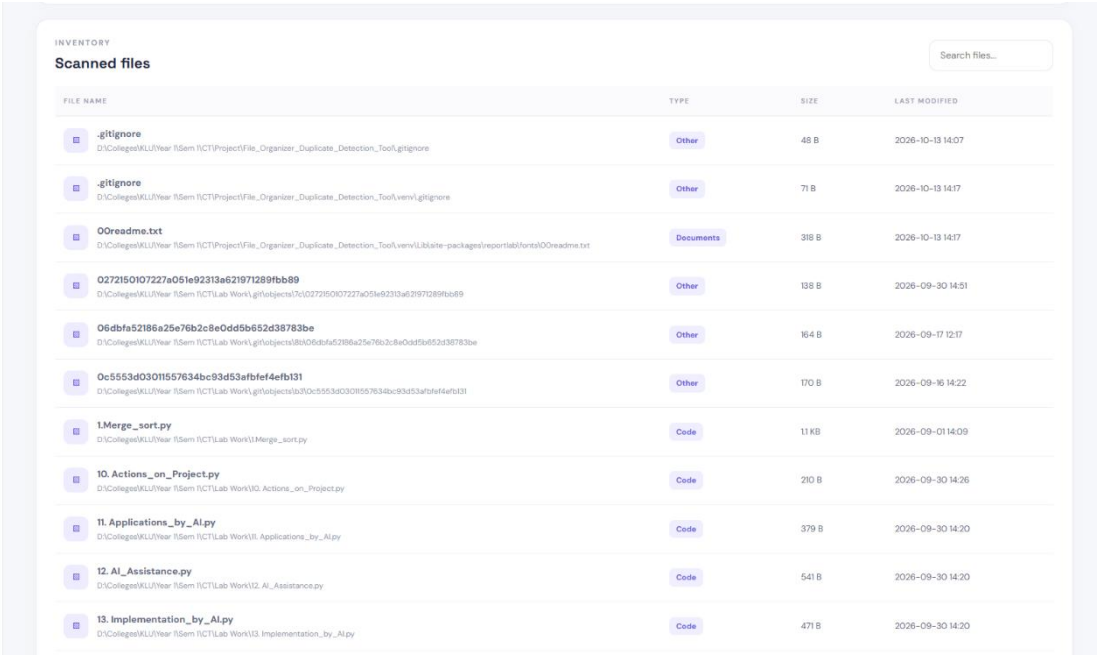
12.4 Safety Recommendations:

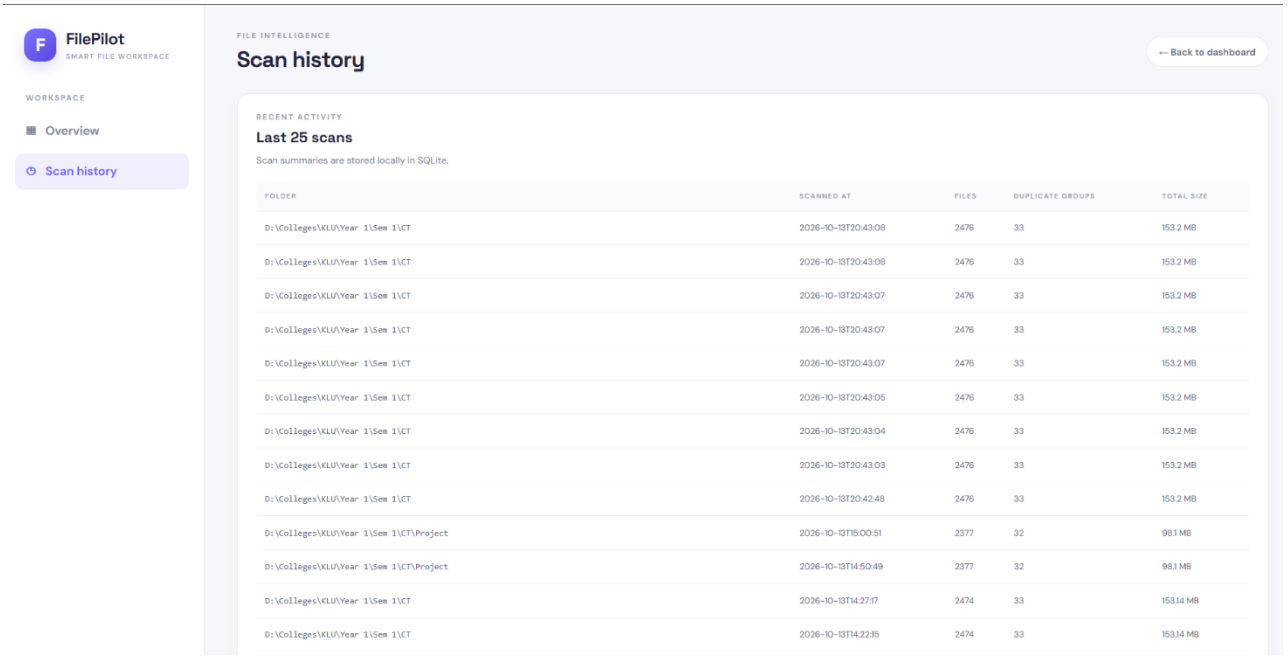
Use disposable test folders before organizing personal data, inspect the preview carefully, maintain backups, and do not expose the local Flask application to untrusted networks. Any future deletion feature should require explicit confirmation and a recovery strategy.

12.5 Result:


FilePilot
SMART FILE WORKSPACE
WORKSPACE
Overview
Duplicate finder
File organizer
Scan history
FILE INTELLIGENCE
Workspace overview
Local workspace
Scan complete: 2476 files checked.
LESS CLUTTER. MORE CLARITY.
Your files,
under control.
Scan folders, spot exact duplicates, and sort files into clean categories — all from one workspace.
PDF
IMG
DOC
01 / SCAN
02 / ANALYZE
03 / ORGANIZE
START HERE
Choose a folder to analyze
Use an absolute path, such as C:\Users\You\Downloads or /home/you/Downloads.
Folder path
D:\Colleges\KLU\Year 1\Sem 1\CT
Scan folder
FILES SCANNED
2476
Across 307 folders
TOTAL SIZE
153.2 MB
Combined file size
DUPLICATE GROUPS
33
104 extra copies found
POTENTIAL SPACE
350.3 KB
Estimated duplicate bytes
SCANNED LOCATION
D:\Colleges\KLU\Year 1\Sem 1\CT


CONTENT MATCHING
Duplicate file groups
33 groups
Files with the same size and SHA-256 hash are treated as exact duplicates.
105.9 KB
3 matching files
pip.exe
pip3.13.exe
pip3.exe
46.9 KB
2 matching files
presentation.pptx
presentation1.pptx
14.2 KB
2 matching files
LICENSE.txt
LICENSE.txt
11.1 KB
2 matching files
LICENSE
LICENSE
9.9 KB
2 matching files
LICENSE.APACHE
LICENSE.APACHE


INVENTORY
Scanned files
Search files...
FILE NAME
TYPE
SIZE
LAST MODIFIED
.gitignore
D:\Colleges\KLU\Year 1\Sem 1\CT\Project\File_Organizer_Duplicate_Detection_Tool\.gitignore
Other
48 B
2026-10-13 14:07
.gitignore
D:\Colleges\KLU\Year 1\Sem 1\CT\Project\File_Organizer_Duplicate_Detection_Tool\.venv\.gitignore
Other
71 B
2026-10-13 14:17
00readme.txt
D:\Colleges\KLU\Year 1\Sem 1\CT\Project\File_Organizer_Duplicate_Detection_Tool\.venv\Lib\site-packages\reportlab\fonts\00readme.txt
Documents
318 B
2026-10-13 14:17
02721501107227a051e92313a621971289fbb89
D:\Colleges\KLU\Year 1\Sem 1\CT\Lab Work\.git\objects\17c\02721501107227a051e92313a621971289fbb89
Other
138 B
2026-09-30 14:51
06dbfa52186a25e76b2c8e0dd5b652d38783be
D:\Colleges\KLU\Year 1\Sem 1\CT\Lab Work\.git\objects\8b\06dbfa52186a25e76b2c8e0dd5b652d38783be
Other
164 B
2026-09-17 12:17
0c5553d03011557634bc93d53afbfef4efb131
D:\Colleges\KLU\Year 1\Sem 1\CT\Lab Work\.git\objects\b3\0c5553d03011557634bc93d53afbfef4efb131
Other
170 B
2026-09-16 14:22
1.Merge_sort.py
D:\Colleges\KLU\Year 1\Sem 1\CT\Lab Work\1.Merge_sort.py
Code
1.1 KB
2026-09-01 14:09
10. Actions_on_Project.py
D:\Colleges\KLU\Year 1\Sem 1\CT\Lab Work\10. Actions_on_Project.py
Code
210 B
2026-09-30 14:26
11. Applications_by_AI.py
D:\Colleges\KLU\Year 1\Sem 1\CT\Lab Work\11. Applications_by_AI.py
Code
379 B
2026-09-30 14:20
12. AI_Assistance.py
D:\Colleges\KLU\Year 1\Sem 1\CT\Lab Work\12. AI_Assistance.py
Code
541 B
2026-09-30 14:20
13. Implementation_by_AI.py
D:\Colleges\KLU\Year 1\Sem 1\CT\Lab Work\13. Implementation_by_AI.py
Code
471 B
2026-09-30 14:20


FilePilot
SMART FILE WORKSPACE
WORKSPACE
Overview
Scan history
FILE INTELLIGENCE
Scan history
← Back to dashboard
RECENT ACTIVITY
Last 25 scans
Scan summaries are stored locally in SQLite.
FOLDER
SCANNED AT
FILES
DUPLICATE GROUPS
TOTAL SIZE
D:\Colleges\KLU\Year 1\Sem 1\CT
2026-10-13T20:43:08
2476
33
153.2 MB
D:\Colleges\KLU\Year 1\Sem 1\CT
2026-10-13T20:43:08
2476
33
153.2 MB
D:\Colleges\KLU\Year 1\Sem 1\CT
2026-10-13T20:43:07
2476
33
153.2 MB
D:\Colleges\KLU\Year 1\Sem 1\CT
2026-10-13T20:43:07
2476
33
153.2 MB
D:\Colleges\KLU\Year 1\Sem 1\CT
2026-10-13T20:43:07
2476
33
153.2 MB
D:\Colleges\KLU\Year 1\Sem 1\CT
2026-10-13T20:43:05
2476
33
153.2 MB
D:\Colleges\KLU\Year 1\Sem 1\CT
2026-10-13T20:43:04
2476
33
153.2 MB
D:\Colleges\KLU\Year 1\Sem 1\CT
2026-10-13T20:43:03
2476
33
153.2 MB
D:\Colleges\KLU\Year 1\Sem 1\CT
2026-10-13T20:42:48
2476
33
153.2 MB
D:\Colleges\KLU\Year 1\Sem 1\CT\Project
2026-10-13T15:00:51
2377
32
98.1 MB
D:\Colleges\KLU\Year 1\Sem 1\CT\Project
2026-10-13T14:50:49
2377
32
98.1 MB
D:\Colleges\KLU\Year 1\Sem 1\CT
2026-10-13T14:27:17
2474
33
153.14 MB
D:\Colleges\KLU\Year 1\Sem 1\CT
2026-10-13T14:22:15
2474
33
153.14 MB

13. CONCLUSION AND FUTURE ENHANCEMENTS

FilePilot demonstrates a practical application of Python programming and computational thinking to everyday file management. It combines filesystem traversal, metadata extraction, SHA-256 hashing, dictionary-based duplicate grouping, extension-based classification, dashboard visualization, safe file organization, and SQLite scan history. Its preview-first approach helps users understand proposed changes before moving files, and its design avoids automatic duplicate deletion.

13.1 Future Enhancements:

- Add a reversible move log and an undo workflow.
- Allow users to filter and export scan reports.
- Introduce background scanning with progress reporting for large directories.
- Add user-controlled duplicate selection and deletion with explicit confirmation and recovery.
- Evaluate near-duplicate image detection as a separate optional feature.
- Add more comprehensive integration tests, code coverage, linting, and static type checks.
- Bundle frontend dependencies locally for improved offline operation.
- Measure performance on representative datasets and document the test environment.

14. REFERENCES

• Project Defence & Demonstration: Computational Thinking and Programming, supplied course document.

• FilePilot project source code: app.py, file_tools.py, templates/, static/, and tests/test_file_tools.py.

• Python documentation: https://docs.python.org/3/

• Flask documentation: https://flask.palletsprojects.com/

• SQLite documentation: https://www.sqlite.org/docs.html

• Chart.js documentation: https://www.chartjs.org/docs/

• Python hashlib documentation: https://docs.python.org/3/library/hashlib.html